Top Secret: HW6 Class Diagram

CS 3140 Team 2-15. Generated from the compiled classes on branch NewDatabaseManagerRefactor, so it matches the current code. Model-View-Controller packages: **org.model** (model), **org.view** (view), **org.controller** (controller), with **org.managers** for database access.

Package overview

org.runtime

Entry point

TopSecret

org.controller

Controller: startup, shared control logic, HTTP server, sessions

AppController *
DebugHelper
LaunchOptions *
LoginHandler *
PageHandler *
ProgramControl
SessionManager *
WebServer *

org.controller.messaging

Controller: agent messaging

MessageController *

org.view

View: terminal menus and HTML template

AgencyInterface *
HtmlPage *

org.view.messaging

View: terminal message display

MessageView *
TerminalMessageView *

org.managers

Data access: SQLite managers, accounts, audit log, files

AgentManager *
AuditLogManager
AuditLogService
AuditLogStore
BriefManager *
CredentialManager
CredentialManagerLibrary
FacilityManager *
FileHandler
IDatabaseManager *
MissionManager *
SqliteAuditLogStore
SqliteUserAccountStore
UserAccountManager
UserAccountStore

org.model

Model: records and value types

Agent
AuditLogEntry
Brief
Cipher
CipherKey
Facility
Mission
RecordType
SearchResult
UserAccount

org.model.messaging

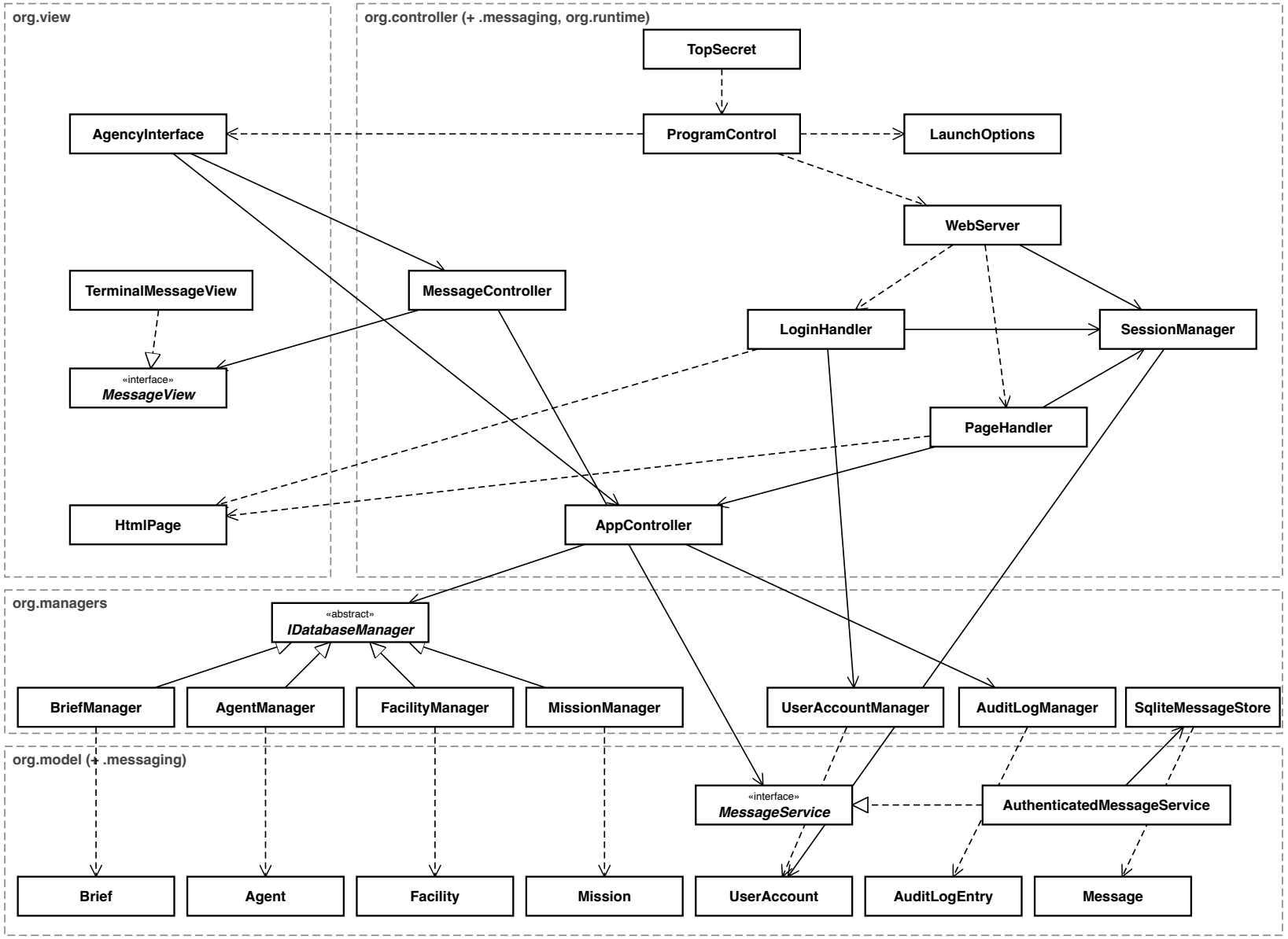
Model: messages and message service

AuthenticatedMessageService *
Message *
MessageService *
MessageStore *
SqliteMessageStore *

* = class added, renamed or split out in HW6 (for example DatabaseManager became IDatabaseManager plus one manager per table, and UserInterface became AgencyInterface). Notation: + public, - private, # protected, ~ package-private; underlined = static; *italic* = abstract or interface. Parameter lists show types only.

Architecture diagram

Solid line, open arrow = association (holds a reference). Dashed line, open arrow = dependency (creates or calls). Solid line, hollow triangle = generalization (extends). Dashed line, hollow triangle = realization (implements). Terminal path: ProgramControl → AgencyInterface. Web path (-web): ProgramControl → WebServer → LoginHandler / PageHandler. Both use AppController for shared control logic.



Relationships

From	Relationship	To
AgentManager	generalization (extends)	IDatabaseManager
BriefManager	generalization (extends)	IDatabaseManager
FacilityManager	generalization (extends)	IDatabaseManager
MissionManager	generalization (extends)	IDatabaseManager
LoginHandler	realization (implements)	HttpHandler
PageHandler	realization (implements)	HttpHandler
AuditLogManager	realization (implements)	AuditLogService
CredentialManagerLibrary	realization (implements)	CredentialManager
SqliteAuditLogStore	realization (implements)	AuditLogStore
SqliteUserAccountStore	realization (implements)	UserAccountStore
UserAccountManager	realization (implements)	CredentialManager
AuthenticatedMessageService	realization (implements)	MessageService
SqliteMessageStore	realization (implements)	MessageStore
TerminalMessageView	realization (implements)	MessageView
AppController	association: briefManager	BriefManager
AppController	association: agentManager	AgentManager
AppController	association: missionManager	MissionManager
AppController	association: facilityManager	FacilityManager
AppController	association: auditLog	AuditLogService
LoginHandler	association: accounts	UserAccountManager
LoginHandler	association: sessions	SessionManager
PageHandler	association: controller	AppController
PageHandler	association: sessions	SessionManager
ProgramControl	association: fileHandler	FileHandler
ProgramControl	association: briefManager	BriefManager
ProgramControl	association: agentManager	AgentManager
ProgramControl	association: facilityManager	FacilityManager
ProgramControl	association: missionManager	MissionManager
ProgramControl	association: auditLogger	AuditLogManager
ProgramControl	association: userManager	UserAccountManager
SessionManager	association: sessions [*]	UserAccount
WebServer	association: controller	AppController
WebServer	association: accounts	UserAccountManager
WebServer	association: sessions	SessionManager
MessageController	association: service	MessageService
MessageController	association: view	MessageView

AuditLogManager	association: store	AuditLogStore
CredentialManagerLibrary	association: fileHandler	FileHandler
CredentialManagerLibrary	association: cipher	Cipher
UserAccountManager	association: accountStore	UserAccountStore
UserAccountManager	association: fileHandler	FileHandler
UserAccountManager	association: cipher	Cipher
Cipher	association: cipherKey	CipherKey
AuthenticatedMessageService	association: agents	AgentManager
AuthenticatedMessageService	association: users	UserAccountStore
AuthenticatedMessageService	association: store	MessageStore
AgencyInterface	association: aManager	AgentManager
AgencyInterface	association: fManager	FacilityManager
AgencyInterface	association: bManager	BriefManager
AgencyInterface	association: uManager	UserAccountManager
AgencyInterface	association: mManager	MissionManager
AgencyInterface	association: auditLog	AuditLogService
AgencyInterface	association: currentUser	UserAccount
AgencyInterface	association: controller	AppController
AgencyInterface	association: messages	MessageController
TopSecret	dependency (creates / uses)	ProgramControl
ProgramControl	dependency (creates / uses)	AgencyInterface
ProgramControl	dependency (creates / uses)	WebServer
ProgramControl	dependency (creates / uses)	AppController
ProgramControl	dependency (creates / uses)	SessionManager
ProgramControl	dependency (creates / uses)	LaunchOptions
WebServer	dependency (creates / uses)	LoginHandler
WebServer	dependency (creates / uses)	PageHandler
LoginHandler	dependency (creates / uses)	HtmlPage
PageHandler	dependency (creates / uses)	HtmlPage
AppController	dependency (creates / uses)	RecordType

package org.runtime

Entry point

TopSecret
+ TopSecret() + main(String[]): void

package org.controller

Controller: startup, shared control logic, HTTP server, sessions

AppController

NEW in HW6

- briefManager: BriefManager
- agentManager: AgentManager
- missionManager: MissionManager
- facilityManager: FacilityManager
- auditLog: AuditLogService

+ AppController(BriefManager, AgentManager, MissionManager, FacilityManager, AuditLogService)
+ listRecords(RecordType): String
+ readRecord(RecordType, int, UserAccount): String
+ auditLog(UserAccount): String
- orNA(String): String

DebugHelper

+ DebugHelper()
+ debugPrintln(Object): void
+ debugPrintArray(Object[]): void

LaunchOptions

NEW in HW6

+ WEB_FLAG: String

- LaunchOptions()
+ isWebMode(String[]): boolean

LoginHandler

NEW in HW6

implements HttpHandler

- accounts: UserAccountManager
- sessions: SessionManager

+ LoginHandler(UserAccountManager, SessionManager)
+ handle(HttpExchange): void
~ parseForm(String): Map<String, String>

PageHandler

NEW in HW6

implements HttpHandler

- controller: AppController
- sessions: SessionManager

+ PageHandler(AppController, SessionManager)
+ handle(HttpExchange): void
- typeFor(String): RecordType
- links(String, String): List<String>
- send(HttpExchange, UserAccount, int, String, String): void

ProgramControl

- fileHandler: FileHandler
- briefManager: BriefManager
- agentManager: AgentManager
- facilityManager: FacilityManager
- missionManager: MissionManager
- auditLogger: AuditLogManager
- userManager: UserAccountManager
- databaseFile: String
- databaseURL: String

+ ProgramControl()
~ ProgramControl(FileHandler, BriefManager, AgentManager, FacilityManager, AuditLogManager, UserAccountManager, MissionManager)
+ run(String[]): void
~ startWeb(int): WebServer
+ getAvailableFiles(): List<String>
+ getFileContents(String): String
+ getFileContents(String, int): String
- getFileContentsWithKey(String, String): String
- createModelTables(): void
~ loadTestData(): void
- importIfEmpty(IDatabaseManager<?>, String): void

SessionManager

NEW in HW6

+ COOKIE_NAME: String
- sessions: Map<String, UserAccount>
- random: SecureRandom

+ SessionManager()
+ createSession(UserAccount): String
+ getUser(String): UserAccount
+ endSession(String): void
+ cookieHeader(String): String
+ tokenFromCookieHeader(String): String

WebServer

NEW in HW6

+ DEFAULT_PORT: int
- controller: AppController
- accounts: UserAccountManager
- sessions: SessionManager
- port: int
- server: HttpServer

+ WebServer(AppController, UserAccountManager, SessionManager, int)
+ start(): void
+ stop(): void
+ getPort(): int
~ send(HttpExchange, int, String): void
~ redirect(HttpExchange, String): void

package org.controller.messaging

Controller: agent messaging

MessageController **NEW in HW6**
- service: MessageService - view: MessageView
+ MessageController(MessageService, MessageView) + listAgents(): void + sendMessage(int, String): void + showInbox(): void

package org.view

View: terminal menus and HTML template

AgencyInterface

NEW in HW6

- aManager: AgentManager
- fManager: FacilityManager
- bManager: BriefManager
- uManager: UserAccountManager
- mManager: MissionManager
- auditLog: AuditLogService
- currentUser: UserAccount
- scanner: Scanner
- output: PrintStream
- controller: AppController
- messages: MessageController
- SEPARATOR: String

+ AgencyInterface(AgentManager, FacilityManager, BriefManager, UserAccountManager, MissionManager, AuditLogService, UserAccount, Scanner, PrintStream)
+ getCurrentUser(): UserAccount
+ run(): void
+ setUser(UserAccount): void
+ displayMainMenu(): void
+ displayTypeMenu(RecordType): void
+ enableMessaging(MessageService): void
- sendAgentMessage(): void
- showMyMessages(): void
+ userCreationPrompt(): String[]
+ changePasswordMenu(): boolean
- promptNewPassword(): String
- promptUntilValid(String, function.Predicate<String>, String): String
- exit(): void
- userCreationMenu(): void
- userPromotionMenu(): void
+ userSelectionPrompt(): String[]
+ message(String): void
- runTypeMenu(RecordType): void
- listRecords(RecordType): void
- readRecord(RecordType): void
- addRecord(RecordType): void
- importRecords(RecordType): Boolean
- addAgent(): Integer
- addMission(): Integer
- displayAuditLog(): void
- isAdmin(): boolean
- readLine(): String
- prompt(String): String
- readId(String): Integer
- invalidSelection(): void
- blankToNull(String): String

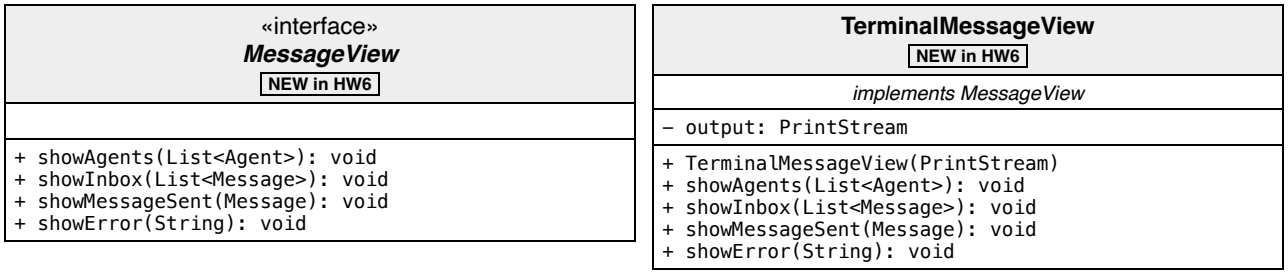
HtmlPage

NEW in HW6

- HtmlPage()
+ page(String, String, boolean, boolean): String
+ list(String, List<String>): String
- isSafeInternalPath(String): boolean
+ paragraphs(String): String
+ loginForm(String): String
+ auditTable(String): String
+ escape(String): String

package org.view.messaging

View: terminal message display



package org.managers

Data access: SQLite managers, accounts, audit log, files

AgentManager

NEW in HW6

extends IDatabaseManager

```
- tableName: String
- columnNames: String[]
```

```
+ AgentManager(String)
+ getURL(): String
+ getTableName(): String
+ getColumnNames(): String[]
+ getRecords(String, String): List<Agent>
+ addRecord(Agent): Agent
+ addRecord(String, String, String, String, int):
Agent
+ updateRecord(String, String, String, String):
Boolean
+ importTSV(String, String, String, String): int
+ createTable(): void
```

AuditLogManager

implements AuditLogService

```
- store: AuditLogStore
```

```
+ AuditLogManager(AuditLogStore)
+ AuditLogManager(String)
+ log(String, String, String, String): void
+ getAuditLog(): List<AuditLogEntry>
- isBlank(String): boolean
- createStore(String): AuditLogStore
```

«interface» *AuditLogService*

```
+ log(String, String, String, String): void
+ getAuditLog(): List<AuditLogEntry>
```

«interface» *AuditLogStore*

```
+ add(AuditLogEntry): void
+ getAll(): List<AuditLogEntry>
```

BriefManager

NEW in HW6

extends IDatabaseManager

```
- tableName: String
- columnNames: String[]
```

```
+ BriefManager(String)
+ getURL(): String
+ getTableName(): String
+ getColumnNames(): String[]
+ getRecords(String, String): List<Brief>
+ addRecord(Brief): Brief
+ addRecord(String, String, String): Brief
+ updateRecord(String, String, String, String):
Boolean
+ importTSV(String, String, String, String): int
+ createTable(): void
```

«interface» *CredentialManager*

```
+ credentialsExist(): boolean
+ isValidUsername(String): boolean
+ isValidPassword(String): boolean
+ verifyLogin(String, String): boolean
+ createCredentials(String, String): boolean
+ changePassword(String, String): boolean
```

CredentialManagerLibrary

implements CredentialManager

```
- fileHandler: FileHandler
- cipher: Cipher
```

```
+ CredentialManagerLibrary(FileHandler, Cipher)
+ credentialsExist(): boolean
+ isValidUsername(String): boolean
+ isValidPassword(String): boolean
+ verifyLogin(String, String): boolean
+ createCredentials(String, String): boolean
+ changePassword(String, String): boolean
```

FacilityManager

NEW in HW6

extends IDatabaseManager

```
- tableName: String
- columnNames: String[]
```

```
+ FacilityManager(String)
+ getURL(): String
+ getTableName(): String
+ getColumnNames(): String[]
+ getRecords(String, String): List<Facility>
+ addRecord(Facility): Facility
+ addRecord(String, String): Facility
+ updateRecord(String, String, String, String):
Boolean
+ importTSV(String, String, String, String): int
+ createTable(): void
```

FileHandler

```
- dataDirectory: String
```

```
+ FileHandler()
+ FileHandler(String)
+ getDataDirectory(): String
+ setDataDirectory(String): void
+ getAvailableFiles(): List<String>
+ readFile(String): String
+ writeFile(String, String): boolean
```

«abstract»
IDatabaseManager<T>
NEW in HW6

```
+ url: String
```

```
+ IDatabaseManager()
+ getURL(): String
+ getTableName(): String
+ getColumnNames(): String[]
+ getRecords(String, String): List<T>
+ addRecord(T): T
+ updateRecord(String, String, String, String):
Boolean
+ importTSV(String, String, String, String): int
+ createTable(): void
+ tableExists(String): Boolean
# connect(): Connection
# insert(String, Object...): int
# update(String, Object...): int
```

MissionManager
NEW in HW6

extends IDatabaseManager

```
- tableName: String
- juncBriefTableName: String
- juncAgentTableName: String
- columns: String[]
```

```
+ MissionManager(String)
+ getURL(): String
+ getTableName(): String
+ getColumnNames(): String[]
+ getRecords(String, String): List<Mission>
- getJuncBriefIds(int): int[]
- getJuncAgentIds(int): int[]
+ addRecord(Mission): Mission
# insert(String, Object...): int
+ addRecord(String, int[], int[], int): Mission
+ updateRecord(String, String, String, String):
Boolean
+ importTSV(String, String, String, String): int
+ createTable(): void
```

SqliteAuditLogStore

implements AuditLogStore

```
- url: String
```

```
+ SqliteAuditLogStore(String)
+ getUrl(): String
+ add(AuditLogEntry): void
+ getAll(): List<AuditLogEntry>
- connect(): Connection
- createTables(): void
```

SqliteUserAccountStore

implements UserAccountStore

```
- url: String
```

```
+ SqliteUserAccountStore(String)
+ getUrl(): String
+ hasUsers(): boolean
+ addUser(UserAccount): boolean
+ getUser(String): UserAccount
+ getUsers(): List<UserAccount>
+ setAdmin(String, boolean): boolean
- connect(): Connection
- createTable(): void
```

UserAccountManager

implements CredentialManager

```
- accountStore: UserAccountStore
- fileHandler: FileHandler
- cipher: Cipher
```

```
+ UserAccountManager(UserAccountStore, FileHandler,
Cipher)
+ UserAccountManager(String, FileHandler, Cipher)
+ isValidUsername(String): boolean
+ isValidPassword(String): boolean
+ credentialsExist(): boolean
+ createCredentials(String, String): boolean
+ hasUsers(): boolean
+ createUser(String, boolean): UserAccount
+ setUserAdmin(String, boolean): boolean
+ createFirstUser(String, String): UserAccount
+ getUser(String): UserAccount
+ getUsers(): List<UserAccount>
+ passwordExists(String): boolean
+ setInitialPassword(String, String): boolean
+ verifyLogin(String, String): boolean
+ changePassword(String, String): boolean
- writePassword(String, String): boolean
- credentialFile(String): String
- wrapPassword(String): String
- createStore(String): UserAccountStore
```

«interface»
UserAccountStore

```
+ hasUsers(): boolean
+ addUser(UserAccount): boolean
+ getUser(String): UserAccount
+ getUsers(): List<UserAccount>
+ setAdmin(String, boolean): boolean
```

package org.model

Model: records and value types

Agent

```
- id: int
- name: String
- dateOfBirth: String
- dateOfDeath: String
- notes: String
- facilityId: int
```

```
+ Agent(int, String, String, String, String, int)
+ getId(): int
+ getName(): String
+ setName(String): void
+ getDateOfBirth(): String
+ setDateOfBirth(String): void
+ getDateOfDeath(): String
+ setDateOfDeath(String): void
+ getNotes(): String
+ setNotes(String): void
+ getFacilityId(): int
+ setFacilityId(int): void
- parseDate(String): LocalDate
```

AuditLogEntry

```
- id: int
- username: String
- action: String
- contentType: String
- contentIdentifier: String
- timestamp: LocalDateTime
```

```
+ AuditLogEntry(int, String, String, String, String,
LocalDateTime)
+ getId(): int
+ setId(int): void
+ getUsername(): String
+ setUsername(String): void
+ getAction(): String
+ setAction(String): void
+ getContentType(): String
+ setContentType(String): void
+ getContentIdentifier(): String
+ setContentIdentifier(String): void
+ getTimestamp(): LocalDateTime
+ setTimestamp(LocalDateTime): void
```

Brief

```
- id: int
- title: String
- date: String
- text: String
- missionId: Integer
```

```
+ Brief(int, String, String, String, Integer)
+ getId(): int
+ getTitle(): String
+ setTitle(String): void
+ getDate(): String
+ setDate(String): void
+ getText(): String
+ setText(String): void
+ getMissionId(): Integer
+ setMissionId(Integer): void
- parseDate(String): LocalDate
```

Cipher

```
- cipherKey: CipherKey
```

```
+ Cipher(CipherKey)
+ cipher(String): String
+ decipher(String): String
```

CipherKey

```
- original: char[]
- ciphered: char[]
```

```
+ CipherKey(String)
+ validateKey(String): boolean
+ getOriginal(): char[]
+ getCiphered(): char[]
```

Facility

```
- id: int
- name: String
- abbreviation: String
```

```
+ Facility(int, String, String)
+ getId(): int
+ getName(): String
+ setName(String): void
+ getAbbreviation(): String
+ setAbbreviation(String): void
```

Mission

```
- id: int
- title: String
- briefIds: int[]
- agentIds: int[]
- facilityId: int
```

```
+ Mission(int, String, int[], int[], int)
+ getId(): int
+ getTitle(): String
+ getBriefIds(): int[]
+ getAgentIds(): int[]
+ getBriefIdsAsString(): String
+ getAgentIdsAsString(): String
+ getFacilityId(): int
+ setFacilityId(int): void
```

«enumeration» RecordType

```
BRIEF
AGENT
FACILITY
MISSION
```

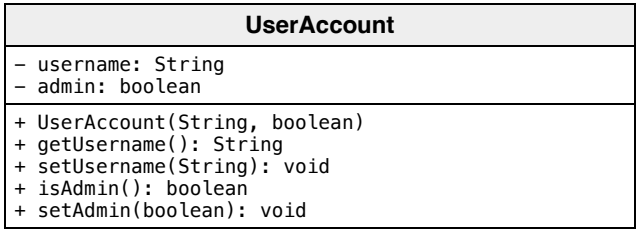
```
- title: String
- singular: String
```

```
- RecordType(String, String)
+ getTitle(): String
+ getSingular(): String
```

SearchResult

```
- matches: List<String>
- message: String
```

```
+ SearchResult(List<String>, String)
+ getMatches(): List<String>
+ getMessage(): String
```


UserAccount
- username: String
- admin: boolean
+ UserAccount(String, boolean)
+ getUsername(): String
+ setUsername(String): void
+ isAdmin(): boolean
+ setAdmin(boolean): void

package org.model.messaging

Model: messages and message service

AuthenticatedMessageService

NEW in HW6

implements MessageService

```
- agents: AgentManager
- users: UserAccountStore
- store: MessageStore
- username: String
```

```
+ AuthenticatedMessageService(AgentManager,
UserAccountStore, MessageStore, UserAccount)
- requireAccount(): void
+ getAgents(): List<Agent>
+ sendMessage(int, String): Message
+ getInbox(): List<Message>
```

Message

NEW in HW6

```
- id: long
- senderUsername: String
- recipientAgentId: int
- recipientName: String
- body: String
- sentAt: LocalDateTime
```

```
+ Message(long, String, int, String, String,
LocalDateTime)
+ getId(): long
+ getSenderUsername(): String
+ getRecipientAgentId(): int
+ getRecipientName(): String
+ getBody(): String
+ getSentAt(): LocalDateTime
```

«interface» *MessageService*

NEW in HW6

```
+ getAgents(): List<Agent>
+ sendMessage(int, String): Message
+ getInbox(): List<Message>
```

«interface» *MessageStore*

NEW in HW6

```
+ save(Message): Message
+ findForRecipient(String): List<Message>
```

SqliteMessageStore

NEW in HW6

implements MessageStore

```
- url: String
```

```
+ SqliteMessageStore(String)
- connect(): Connection
+ save(Message): Message
+ findForRecipient(String): List<Message>
```